# Backend Optimizer

**PROPOSAL FOR COMPETITION: BACKEND OPTIMIZER**

## Introduction

Backend Optimizer is a systems-focused programming competition in which participants are given a predefined backend specification for a Reddit/Twitter-like social feed application. The challenge is to build an implementation that is fully correct while achieving the highest possible performance under heavy read and write load.

## Competition Structure

The competition will be conducted in a single round with the following specifications:

### Problem Statement

Participants will be given a backend problem statement describing a social feed system through a set of required API endpoints, request formats, response formats, and correctness constraints.

Every post may optionally contain a `parent_post_id`. If present, that post is treated as a comment on the parent post. Comments are therefore represented using the same post model as normal posts.

Each team must implement the backend so that:

- Every endpoint returns the exact expected output
- The system handles concurrent requests correctly
- The system performs as efficiently as possible under an automated benchmark

Participants are free to choose:

- Programming language
- Framework
- Database
- Caching layer
- Internal architecture

### Submission Format

- Each team must submit a Docker image of the complete backend system
- If additional services are required, participants may either bundle them into the image or submit a Docker Compose setup, depending on final infrastructure policy
- A short README must explain how to run the service and expose the required port(s)

## Verification and Benchmarking

All submissions will be tested on standardized hardware using a common evaluation harness.

The organizers will build a "backend obliterator" that:

- Spins up the submitted backend image
- Sends a predefined set of HTTP requests to all required endpoints
- Verifies correctness of every response
- Sends heavy read and write traffic at the fastest sustainable rate possible
- Executes parallelizable queries in parallel
- Measures requests per second, latency, and memory usage

Only correct responses will be considered for scoring.

The benchmark workload will primarily focus on:

- Creating users and posts
- Reading user profiles and post lists
- Fetching the home/feed timeline under high concurrency
- Updating social state such as likes while preserving correctness

## Evaluation Criteria

- **Correctness**: All API responses must exactly match expected outputs, even under concurrent reads and writes
- **Performance**: Primary ranking metric based on throughput and latency under benchmark load
- **Memory Usage**: Lower memory consumption will be considered in scoring
- **System Design**: Clean and reliable engineering choices may be used as a tie-breaker

Bonus points for:

- Efficient handling of concurrent workloads

- Smart caching, indexing, and feed-generation strategies
- Creative but reliable architectural optimizations

## Team Composition

- 4 members per team

## Languages

Participants are free to use any programming language or framework of their choice, provided the final submission can be executed in Docker.

## `POST /auth/register`

Registers a new user account.

Authentication:

- Does not require authentication.

Headers:

- `Content-Type: application/json`

Request body:

```
{
  "username": "alice",
  "password": "password123",
  "display_name": "Alice"
}
```

Response body:

```
{
  "user_id": "u_123",
  "username": "alice",
  "display_name": "Alice",
  "token": "jwt-or-session-token"
}
```

## `POST /auth/login`

Authenticates an existing user and returns an auth token.

Authentication:

- Does not require authentication.

Headers:

- `Content-Type: application/json`

Request body:

```
{
  "username": "alice",
  "password": "password123"
}
```

Response body:

```
{
  "user_id": "u_123",
  "token": "jwt-or-session-token"
}
```

## `GET /user/details`

Returns the profile details of the authenticated user, or of a requested user if a `user_id` query parameter is provided.

Authentication:

- Requires authentication.

Headers:

- `Authorization: Bearer <token>`

Request:

- Query params optional: `user_id`
- Example: `/user/details?user_id=u_123`

Request body:

```
null
```

Response body:

```
{
  "user_id": "u_123",
  "username": "alice",
  "display_name": "Alice",
  "post_count": 42
}
```

## `POST /user/delete`

Deletes the authenticated user account.

Authentication:

- Requires authentication.

Headers:

- `Authorization: Bearer <token>`
- `Content-Type: application/json`

Request body:

```
{}
```

Response body:

```
{
  "success": true
}
```

## `GET /user/get_posts`

Returns posts created by a given user, ordered from newest to oldest.

Authentication:

- Requires authentication.

Headers:

- `Authorization: Bearer <token>`

Request:

- Query params: `user_id`, optional pagination params such as `cursor` and `limit`
- Example: `/user/get_posts?user_id=u_123&limit=20&cursor=cursor_1`

Request body:

```
null
```

Response body:

```
{
  "posts": [
    {
      "post_id": "p_1",
      "author_id": "u_123",
      "content": "hello world",
      "parent_post_id": null,
      "created_at": "2026-03-29T10:00:00Z",
      "like_count": 10,
      "comment_count": 2,
      "liked_by_me": false
    }
  ],
  "next_cursor": "cursor_2"
}
```

`comment_count` counts only direct child posts whose `parent_post_id` equals this post's ID. It does not include deeper descendants.

## `GET /user/liked_posts`

Returns posts liked by a given user.

Authentication:

- Requires authentication.

Headers:

- `Authorization: Bearer <token>`

Request:

- Query params: `user_id`, optional pagination params such as `cursor` and `limit`
- Example: `/user/liked_posts?user_id=u_123&limit=20&cursor=cursor_1`

Request body:

```
null
```

Response body:

```
{
  "posts": [
    {
      "post_id": "p_7",
      "author_id": "u_999",
      "content": "a liked post",
      "parent_post_id": null,
      "created_at": "2026-03-29T09:00:00Z",
      "like_count": 30,
      "comment_count": 5,
      "liked_by_me": true
    }
  ],
  "next_cursor": "cursor_3"
}
```

`comment_count` counts only direct child posts whose `parent_post_id` equals this post's ID. It does not include deeper descendants.

## `POST /posts/create`

Creates a new post with optional media. If `parent_post_id` is provided, the created post is a comment. This is a heavy-write endpoint.

Authentication:

- Requires authentication.

Headers:

- `Authorization: Bearer <token>`
- `Content-Type: multipart/form-data`

Request:

- Multipart form submission.
- Form fields:
  - `content`: text content of the post
  - `parent_post_id`: optional parent post ID; if present, the post is a comment
  - `media[]`: zero or more uploaded files as raw bytes

Example form submission:

```
content = "Check out my setup!"
parent_post_id = "p_100"   # optional
media[] = <image bytes>
media[] = <video bytes>
```

Response body:

```
{
  "post_id": "p_123",
  "author_id": "u_123",
  "content": "Check out my setup!",
  "parent_post_id": null,
  "media": [
    {
      "type": "image",
      "media_id": "m_001"
    },
    {
      "type": "video",
      "media_id": "m_002"
    }
  ],
  "created_at": "2026-03-29T10:10:00Z"
}
```

## `GET /posts/details`

Returns full details for a single post. This endpoint is used by the tester to verify that media was saved correctly.

Authentication:

- Requires authentication.

Headers:

- `Authorization: Bearer <token>`

Request:

- Query params: `post_id`
- Example: `/posts/details?post_id=p_123`

Request body:

```
null
```

Response body:

```
{
  "post_id": "p_123",
  "author_id": "u_123",
  "content": "Check out my setup!",
  "parent_post_id": null,
  "created_at": "2026-03-29T10:10:00Z",
  "like_count": 5,
  "comment_count": 2,
  "liked_by_me": false,
  "media": [
    {
      "type": "image",
      "media_id": "m_001",
      "url": "/media/m_001.jpg",
      "thumbnail_url": "/media/m_001_thumb.jpg"
    }
  ]
}
```

`comment_count` counts only direct child posts whose `parent_post_id` equals this post's ID. It does not include deeper descendants.

## `POST /posts/delete`

Deletes a post created by the authenticated user.

Authentication:

- Requires authentication.

Headers:

- `Authorization: Bearer <token>`
- `Content-Type: application/json`

Request body:

```
{
  "post_id": "p_123"
}
```

Response body:

```
{
  "success": true
}
```

**Tester Validation Logic:** The benchmark tool will attempt to download the file at `GET <Your-Backend-URL>/media/m_001.jpg`.

1. If the request returns a **404**, the submission is marked **Incorrect**.
2. The downloaded bytes will be hashed (MD5) and compared against the original upload. A mismatch results in a **Correctness Penalty**.

## `POST /posts/like`

Likes or unlikes a post. This should be concurrency-safe under heavy write load.

Authentication:

- Requires authentication.

Headers:

- `Authorization: Bearer <token>`
- `Content-Type: application/json`

Request body:

```
{
  "post_id": "p_123",
  "liked": true
}
```

Response body:

```
{
  "post_id": "p_123",
  "like_count": 11,
  "liked_by_me": true,
}
```